

MAPLE: Encoding Dexterous Robotic Manipulation Priors Learned From Egocentric Videos

Alexey Gavryushin¹✉ Xi Wang¹ Robert J. S. Malate^{1,2} Chenyu Yang¹ Xiangyi Jia¹
 Shubh Goel¹ Davide Liconti¹ René Zurbrügg¹ Robert K. Katzschmann^{1,2} Marc Pollefeys^{1,3}

¹ETH Zürich, Rämistrasse 101, 8092 Zürich, Switzerland

²Mimic Robotics, Andreasstrasse 5, 8050 Zürich, Switzerland

³Microsoft Research, Seestrasse 356, 8038 Zürich, Switzerland

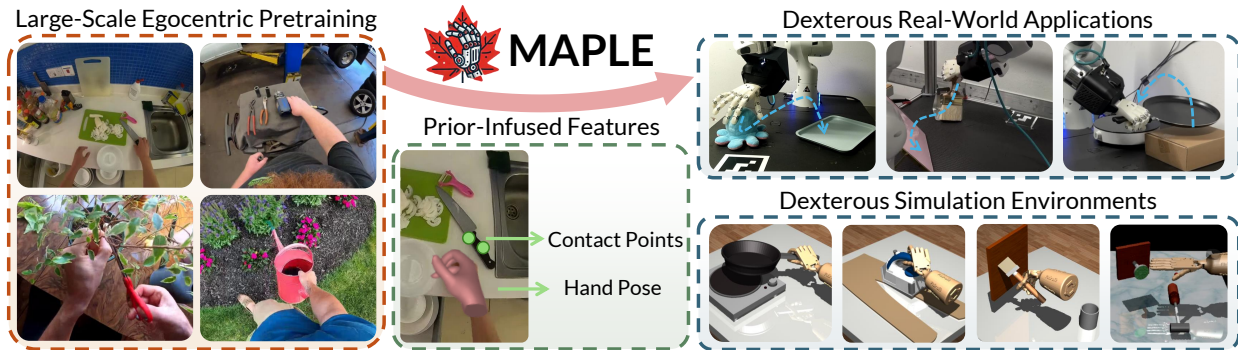


Figure 1. **MAPLE: Encoding Dexterous Robotic Manipulation Priors Learned From Egocentric Videos.** We present MAPLE, a framework that learns dexterous manipulation priors from egocentric videos and produces features well-suited for downstream dexterous robotic manipulation tasks. Experiments in both simulation and real-world settings demonstrate that MAPLE enables efficient policy learning and improves generalization across various tasks.

Abstract

Large-scale egocentric video datasets capture diverse human activities across a wide range of scenarios, offering rich and detailed insights into how humans interact with objects, especially those that require fine-grained dexterous control. Such complex, dexterous skills with precise controls are also crucial for many robotic manipulation tasks, which are often insufficiently addressed by traditional data-driven approaches to robotic manipulation. To address this gap, we leverage manipulation priors learned from large-scale egocentric videos to improve policy learning for dexterous robotic manipulation tasks. We present MAPLE, a novel method for dexterous robotic manipulation that exploits rich manipulation priors to enable efficient policy learning and better performance on diverse, complex manipulation tasks. Specifically, we predict hand-object contact points and detailed hand poses at the moment of contact and use the learned features to train policies for downstream manipulation tasks. Experimental results demonstrate the effective-

ness of MAPLE across existing simulation benchmarks, as well as a newly designed set of challenging simulation tasks, which require fine-grained object control and complex dexterous skills. Our newly designed simulation environments address the shortage of dexterous manipulation benchmarks in the literature. The benefits of MAPLE are further highlighted in real-world experiments using a dexterous robotic hand, while simultaneous evaluation across both simulation and real-world experiments has often been underexplored in prior work. Our code, trained model, and the new manipulation benchmark suite will be made publicly available on <https://algvr.com/maple/>.

1. Introduction

As robotic systems become increasingly deployed across various domains, dexterous manipulation emerges as a critical capability, going beyond basic automation to enable robots to augment human abilities and function autonomously in dynamic, real-world environments. Consider a household robot tasked with preparing a simple meal, such as spaghetti with tomato sauce. To complete the task, the

✉ Correspondence to agavryushin@ethz.ch

robot must interact with various kitchen objects—locating a pan, filling it with water, and placing it on the stove to boil. In such scenarios, the robot encounters a wide range of everyday objects, many of which require complex dexterous manipulation skills. Robust dexterous manipulation priors are essential for successfully performing these tasks in the real world. What is then the best source of information for teaching robots dexterous manipulation skills?

Recently, the computer vision community has shown a growing interest in the egocentric perspective [8, 14, 15, 25, 62], as it gives unique insights into human interactions with objects. Large-scale egocentric datasets, in particular, capture the complex, dexterous dynamics of hand-object interactions across a wide range of object categories. It thus suggests itself that these datasets are well-suited for learning object manipulation priors directly from natural human-object interactions. Several recent studies [31, 39, 52, 54] have explored this direction by training visual representations on large collections of human egocentric videos. However, these approaches generally lack a specific focus on dexterous manipulation and often struggle to generalize, particularly in human-centric environments where fine-grained control is essential [3, 10, 45, 46].

Why has learning generalizable dexterous manipulation priors remained so challenging? While general-purpose representations trained on large datasets capture rich scene-level details [4, 44], they lack a specific focus on manipulation-relevant cues, making them suboptimal. Other self-supervised learning approaches (e.g. VIP [30] trained with contrastive losses) struggle to effectively extract dexterous manipulation information from egocentric videos, especially due to the complex and cluttered nature of real-world scenarios depicted in these videos. This raises a key question: what specific information should a visual encoder designed for object manipulation learn to extract? We hypothesize that low-level interaction cues, specifically *object contact points* and *grasping hand poses*, could provide a strong prior for downstream dexterous manipulation tasks. To this end, we introduce MAPLE, a novel method that learns dexterous manipulation priors from egocentric videos for robotic manipulation.

Our key insight is to design learning objectives that guide representations to effectively capture dexterous manipulation knowledge. More specifically, we focus on training a visual encoder aimed at extracting where and how to interact with objects during manipulation. We use existing state-of-the-art methods to automatically extract supervision signals for training, greatly reducing the cost of dataset acquisition. Given a single image of the target object, MAPLE learns to predict hand-object contact points and the 3D hand pose at the moment of contact. The learned visual representations are then used as input to policy networks trained to control dexterous robot hands.

We demonstrate that MAPLE enables the extraction of features from egocentric videos that are well-suited for robotic manipulation tasks requiring complex dexterous skills. We conduct experiments on established simulation-based benchmarks, specifically designed to evaluate dexterous manipulation skills. Additionally, we evaluate MAPLE on a newly designed set of challenging simulated manipulation tasks, which require more fine-grained object control and advanced dexterous skills. A comprehensive evaluation is performed across existing and newly proposed dexterous manipulation tasks, comparing MAPLE against general-purpose visual encoders and state-of-the-art methods. The benefits of MAPLE are further highlighted in real-world experiments using a dexterous robotic hand. Note that such an evaluation setting using both simulated and real-world experiments has been largely underexplored in prior work. We will publicly release our codebase, data extraction pipeline, extracted data, and the newly designed benchmark to support future research. In summary, our contributions are three-fold:

- We propose MAPLE, a novel visual encoder designed for dexterous manipulation tasks by learning to predict low-level cues for object manipulation.
- We propose a new set of custom-designed dexterous manipulation tasks in simulation that require more fine-grained object control.
- We demonstrate that MAPLE improves performance on downstream dexterous robotic manipulation tasks in both simulated environments and real-world settings, highlighting its improved generalization ability.

2. Related Work

Our work focuses on dexterous manipulation in robotics. In the following, we review the most relevant research areas.

Dexterous Manipulation. Dexterous manipulation describes the manipulation of objects with multi-fingered hands. Due to the high-dimensional action space and complex physical interactions, it is one of the most challenging tasks in robotics. Traditional approaches in dexterous manipulation typically rely on analytical grasp planning methods that use models [6] or optimization [59, 61]. These grasp proposals are then executed in an open loop, feed-forward fashion using a suitable task-space controller. While this modular approach allows for more flexibility, it faces challenges when dealing with partially observed environments or when adaptability and tight coupling of vision and control are necessary. Therefore, more recent work has investigated the use of data-driven end-to-end techniques such as reinforcement learning to learn manipulation policies [42, 66, 67]. Reinforcement learning provides a powerful tool to learn robust dexterous manipulation policies, but its performance heavily relies on simulation accuracy as well as proper reward formulations, which might be chal-

lenging to formulate. This has sparked research interest that led to the advent of imitation learning [5, 21, 27, 68], which directly leverages human demonstrations as a supervision signal. While most of these methods are able to learn meaningful manipulation policies, their generalization across different task is still limited – often relying on a large amount of task-specific trajectories in order to be used for new tasks.

Learning From Egocentric Videos. Large-scale human-recorded visual observations have been widely used to extract embeddings or motion priors for robotic manipulation. Existing methods can largely be divided into implicit and explicit methods.

Implicit methods [30, 31, 34, 39] utilize large-scale contrastive learning to derive meaningful features from egocentric interaction data. Some works, such as [30, 31, 64] leverage large-scale video datasets to learn goal-conditioned value functions [30] or train masked autoencoders (MAE) [31, 64]. Additionally, methods like [39] integrate language embeddings by applying a contrastive loss across image and language modalities. However, due to their implicit nature, these methods can’t be directly used for robotic manipulation. Instead, their pre-trained (frozen) embeddings serve as inputs for reinforcement learning [64] or imitation learning [39] algorithms.

Conversely, *explicit methods* [1, 2, 20, 33, 38] aim to directly learn interaction affordances from videos by using human interaction as an explicit supervision signal. These approaches typically predict interaction trajectories [38], contact heatmaps [1, 20, 33], or future contact points and wrist position [54]. While their output could be directly used within a task-specific controller, most methods use the final predictions or intermediate embeddings in policy learning due to the complexity of the manipulation task. Although explicit supervision helps learn a better-suited latent space, most existing approaches focus on high-level interactions and are primarily designed for simple two-finger manipulation. Furthermore, they often depend on human-annotated data, limiting scalability. In contrast, our method does not rely on any human-annotated data and adopts a more fine-grained formulation capturing detailed manipulation dynamics, well-suitable for dexterous tasks.

Evaluation in Robotic Simulators. We categorized simulation benchmarks by the type of end-effector used. Benchmarks like MetaWorld [65], Franka Kitchen [13, 16], and Habitat 2.0 [55] focus on parallel jaw grippers, defining tasks such as object manipulation, cooking-related activities, and home environment rearrangement tasks. The DeepMind Control Suite [56] also includes basic object manipulation tasks in abstract environments. For three-finger grippers, TriFinger [63] provides a benchmark for object manipulation, particularly evaluating toy cube tasks in both real and simulated settings. Finally, dexterous hand benchmarks offer more complex manipulation challenges. DAPG

[46] introduces four environments for a simulated Adroit [23] hand, while DexMV [43] includes tasks like pouring, object relocation, and target-based movement, requiring more fine-grained control. A key limitation across many benchmarks is their reliance on low-DoF grippers, restricting their ability to evaluate policies for fine-grained manipulation. This motivates us to design a more comprehensive dexterous manipulation benchmark.

Evaluation in Real World. In the robotics community, imitation learning is becoming a standard approach to performing dexterous manipulation tasks [5, 11, 68], even driving the development of foundation model-style generalist robot policies that can be fine-tuned with target data from small samples [22, 57]. This is because it enables learning from task demonstrations without the need for simulation environments or task-specific rewards. However, these models have been primarily trained and deployed on low-DoF grippers. Furthermore, for dexterous manipulators, there are no standard datasets due to the high variation between designs. This motivates us to design dexterous manipulation tasks in the real world, reproducible with a commercially available dexterous hand [35].

3. MAPLE

We present *Encoding Manipulation Priors Learned from Egocentric Videos* (MAPLE), an approach for learning manipulation priors from egocentric videos to enable dexterous robotic manipulation. The MAPLE pipeline involves solving two key problems: how to model manipulation priors and extract corresponding labels from egocentric videos, and how to learn these priors from the extracted labels. Figure 2 illustrates the entire pipeline.

3.1. Extracting Manipulation Labels From Videos

MAPLE aims to learn features useful for dexterous manipulation from large, publicly available egocentric video datasets. To effectively represent manipulation priors, we consider the contact points between hands and objects, along with the corresponding 3D hand poses at the moment of contact. We use off-the-shelf tools to automatically parse these hand-object interactions and extract the relevant training labels.

Specifically, we define a *contact frame* f_c , which marks the moment when the hand first makes contact with the object. We also define a *prediction frame* f_p , occurring earlier in the video than f_c , when the hand is sufficiently distant from the object it is about to interact with. From the contact frame f_c , we extract the hand pose vector $\mathcal{H}_c$ representing the interacting hand. Using $\mathcal{H}_c$, we further extract the contact points pt_c and then track back to f_p to obtain the future contact points pt_p on the prediction frame. Figure 3 provides a visualization of our label extraction procedure.

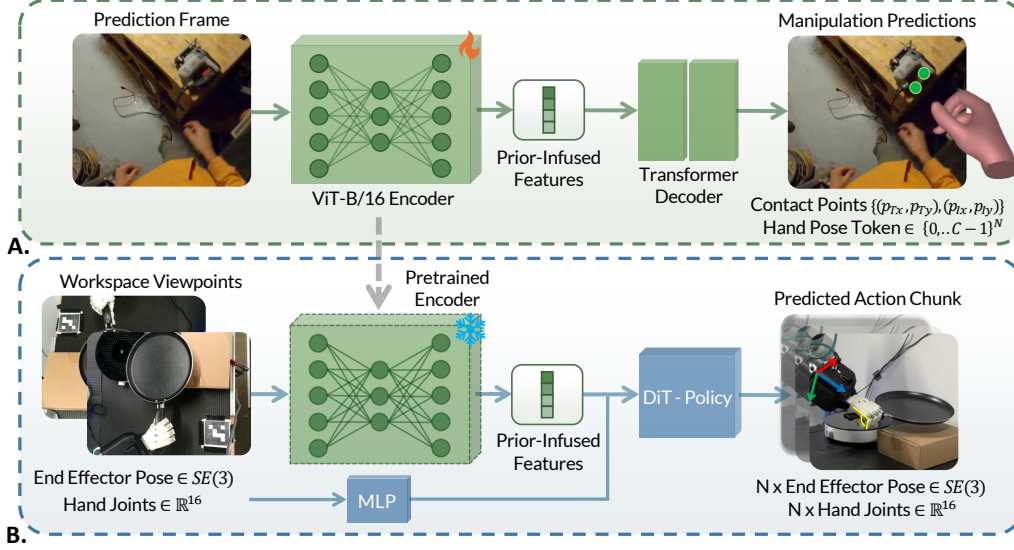


Figure 2. **Overview of MAPLE.** Given a single input frame, the encoder is trained to reason about hand-object interactions, specifically predicting contact points and grasping hand poses. This training infuses a **manipulation prior** into the learned feature representation, making it well-suited for downstream robotic manipulation. Features extracted from the frozen visual encoder, combined with robotic hand positions, are fed into a Transformer-based diffusion policy network to predict dexterous hand action sequences.

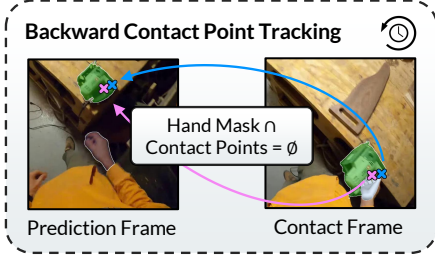


Figure 3. **Extracting Contact Points and Hand Poses From Egocentric Videos.** We first extract contact points and hand poses from the contact frame, and use a point tracker to backtrack the contact locations on the prediction frame.

Extract Contact Supervision From the Contact Frame.

To locate moments of contact, we process all frames in the dataset using an off-the-shelf hand-object contact segmentation model [9] and consider the first contact frame as f_c . To avoid extracting identical f_p from neighboring contact frames, we only process contact frames with no other f_c preceding in the last $\delta_{min} = 90$ frames.

We define *contact points* pt_c as the points on the object where the fingertips of the thumb and index finger first make contact with the object. For each contact frame, we use a pre-trained hand pose estimation model [41] to extract the pose vector of the interacting hand, resulting in MANO [49] hand poses $\mathcal{H}_c \in \mathbb{R}^{21 \times 3}$. The extracted $\mathcal{H}_c$ are forwarded through a MANO layer to obtain 2D fingertip keypoints. We then project the thumb and index fingertip keypoints onto the object mask to obtain pt_c . When calculating the position of the projected fingertip points, we use a slightly smaller object mask from which a portion of the boundary

has been erased by $n_e = 12$ binary erosion [51] operations. This helps when finding the contact points in the prediction frame in the next step. Details including visualizations and failure analysis are provided in Supp. Mat.

Extract Supervision From the Prediction Frame. To identify the prediction frame f_p , we run a point tracker [47] to project the contact points backward in video time until we find the first frame where the hand mask does not yet intersect the object mask. Using smaller object masks for contact point identification helps prevent the point tracker from tracking the fingers of the manipulating hand instead of the object. When checking for the intersection, we expand the object mask by $n_d = 75$ binary dilation [51] iterations to ensure a sufficient distance from the object to the hand. This helps make the contact point regression less trivial than simply locating the object immediately next to the hand. Prior work [1, 54] randomly samples contact points from the intersection of the bounding boxes induced by the hand and the manipulated object, which is highly prone to erroneous results, such as when points are sampled from the background as the hand is touching a concave object. In contrast, we project fingertip locations to the manipulated objects’ segmentation masks to extract precise contact information.

Tokenizing Hand Poses. Learning to predict hand poses associated with grasping a target object is a challenging task, due to the requirement of conforming to anatomical constraints and reasoning about a complex kinematic tree with a high number of joints. Furthermore, a large subset of the joint configuration space results in hand poses exhibiting self-penetration. We simplify the prediction task

by reframing it as a *classification* task and letting the model select the hand pose among a limited list of “reasonably frequent” (i.e., observed in videos) and anatomically correct choices rather than regress the correct hand pose by itself. For that purpose, we train a hand pose tokenizer on hand poses extracted by HaMeR [41] from a randomly selected subset of the Ego4D dataset. We use the same Mem-codes [32]-based tokenization scheme as the human pose tokenizer proposed in 4M [36], employing 8 codebooks of size 1024 each to represent a single MANO [49] hand pose. Qualitative reconstruction examples of the hand tokenizer can be found in the Supp. Mat.

3.2. Encoding Manipulation Priors

We aim to train a visual encoder tailored for dexterous manipulation by reasoning about human hand-object interactions extracted from egocentric videos. We use an encoder-decoder structure as a stepping stone in learning useful representations: the encoder processes input prediction frames into embedding vectors, which the decoder then uses to predict contact points and future hand poses. The decoder relies entirely on the encoder’s embeddings, which have a much lower dimensionality than the input images, to infer its output. This bottleneck design encourages the encoder to produce information tailored specifically for object manipulation-related predictions, rather than learning to produce “general-purpose” visual embeddings [4, 44].

Contact Point Prediction. We employ the cross-entropy loss to learn to predict contact points pt_p in the prediction frame f_p . To do so, we discretize the image’s width w and height h into B_x resp. B_y bins each and let the decoder predict the respective bin indices into which the contact point should fall. We set $B_x = 100$ and $B_y = 100$ in our experiments. We formulate the contact loss $\mathcal{L}_{con,pt}$ for one sample and one contact point pt as the following:

$$\mathcal{L}_{con,pt_p} = CE(\arg \max_{j \in \{1, \dots, B_x\}} \tilde{p}_{pt,x,j}, \left\lfloor \frac{\hat{x}}{w} \times B_x \right\rfloor) + CE(\arg \max_{j \in \{1, \dots, B_y\}} \tilde{p}_{pt,y,j}, \left\lfloor \frac{\hat{y}}{h} \times B_y \right\rfloor),$$

where $\tilde{p}_{pt,x} \in \mathbb{R}^{B_x}$, $\tilde{p}_{pt,y} \in \mathbb{R}^{B_y}$ are the decoder’s corresponding logit vectors. $CE(\tilde{c}, \hat{c})$ represents the cross-entropy loss for the predicted class $\tilde{c}$ and the ground-truth class $\hat{c}$. $\hat{x}$ and $\hat{y}$ are the extracted contact point coordinates. We use $pt \in \{0, 1\}$ for the thumb and index contact points.

Hand Pose Prediction. During MAPLE’s training, we let the decoder predict a probability distribution for each hand pose token in the token sequence, then use the highest-scoring token logit to determine the final token from which to reconstruct the hand pose. Similarly to the contact point prediction, we also use a cross-entropy loss for this. Specifically, we process a MANO hand pose $\mathcal{H} \in \mathbb{R}^{21 \times 3}$ using a

tokenization procedure

$$\mathcal{T} : \mathbb{R}^{21 \times 3} \mapsto \{0, 1, \dots, C - 1\}^N,$$

which converts the pose into a token t_n , where $1 \leq n \leq N$ and C represents the codebook size of the tokenizer. Let $\tilde{p}_n$ be the decoder’s logit predictions for estimating t_n . Then the loss formulation $\mathcal{L}_{hand}$ is given by:

$$\mathcal{L}_{hand} = \sum_{n=1}^N CE(\arg \max_{j \in \{1, \dots, C\}} \tilde{p}_{n,j}, t_n).$$

Ensuring Temporal Distinctiveness. We hypothesize that the temporal distinctiveness of the input features plays an important role in the success of policy learning. Note that our initial weights are copied from DINO [4], which provides a highly distinctive visual prior due to DINO’s contrastive training objectives. To maintain this distinctiveness, we can optimize over a small set of parameters, e.g. those of the LayerNorm [24] modules of the ViT [12] backbone, which would change the representation less aggressively and preserve the prior distinctiveness. Alternatively, we can employ a temporal contrastive loss [40] during training to maintain distinctiveness in the face of more drastic changes. Both approaches have been explored in prior work [39, 54] and are incorporated in MAPLE as **MAPLE-LN**, where we train only the LayerNorm parameters, and **MAPLE-AP**, where all parameters are trained.

For a given frame $f_j^{v_m}$ at position j of video v_m , we follow the InfoNCE objective employed in [39], pushing representations of frames from different videos ($f_j^{v_m}$ and $f_\ell^{v_n}$, $v_m \neq v_n$) or farther in time ($f_i^{v_m}$ and $f_k^{v_m}$, $i < j < k$) apart while pulling representations from close frames in the same video ($f_i^{v_m}$ and $f_j^{v_m}$, $i < j$) together:

$$\mathcal{L}_{ctr} = -\log \left(\frac{e^{s(f_i^{v_m}, f_j^{v_m})}}{e^{s(f_i^{v_m}, f_j^{v_m})} + e^{s(f_i^{v_m}, f_k^{v_m})} + e^{s(f_j^{v_m}, f_\ell^{v_n})}} \right).$$

s is a function increasing with the similarity of its input vectors, such as $s(x, y) = -\|x - y\|_2^2$.

The overall loss for a single sample is thus calculated as

$$\mathcal{L} = \sum_{pt \in \{0, 1\}} \mathcal{L}_{con,pt_p} + \lambda_{hand} \mathcal{L}_{hand} + \lambda_{ctr} \mathcal{L}_{ctr}.$$

We simply set $\lambda_{ctr} = 1$ for MAPLE-AP and $\lambda_{ctr} = 0$ for MAPLE-LN, as well as $\lambda_{hand} = 1$ for both models.

Implementation Details. Similar to other ViT [12]-derived baselines [4, 31, 54], we employ a ViT-B/16 as the encoder’s architecture. The encoder is initialized with the weights of [4] and produces visual features of dimension $d = 768$. To ensure fair comparisons and evaluate the contribution of our supervised training procedure, rather than the choice of architecture, we do not opt for a larger or more

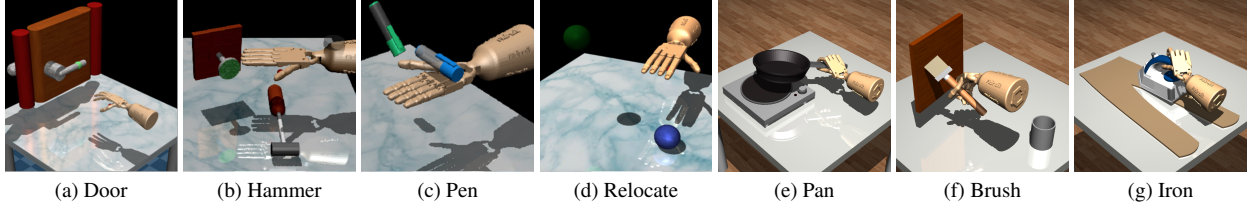


Figure 4. **Simulated Evaluation Environments.** We evaluate our method on four environments from DAPG [46] (a-d) and propose three new robotic environments (e-g). Our new environments aim to evaluate the manipulation capabilities of a set of objects commonly used by humans, namely a brush, a clothes iron, and a pan.

intricate backbone. We use a Transformer [60] decoder model with 2 Transformer layers and 4 attention heads per layer. The encoder and decoder are jointly trained using the AdamW [29] optimizer with a learning rate of 5×10^{-5} . We use a batch size of 128 when training MAPLE-LN, and 512 when training MAPLE-AP.

4. Experiments

In our experiments, we evaluate the effectiveness of MAPLE for robotic dexterous manipulation tasks in both simulation and real-world environments.

4.1. Experimental Setup

Training Data. We run our extraction pipeline on *Ego4D* [14], which consists of more than 3,000 hours of unscripted human activities across diverse scenarios (e.g., gardening, exercise, and cooking). To ensure a fair comparison, we follow the established protocols as in [39, 54] and select the same subset covering approximately 1,590 hours of video for extracting data. Our pipeline produces approximately 82,100 training samples from this subset. See the Supp. Mat. for more details.

Baseline Encoders. We evaluate against both the commonly used, general-purpose visual encoder DINO [4], as well as four state-of-the-art (SotA) encoders designed specifically for robotic manipulation:

- **DINO** [4]: A self-supervised visual encoder trained with a contrastive loss on images and their extracted patches.
- **R3M** [39]: A robotic manipulation baseline using a temporal contrastive loss on images and another contrastive loss estimating task completion based on video summaries matched with visual features for pairs of images.
- **HRP** [54]: A visual encoder trained for robotic manipulation settings using contact points extracted from hand-object bounding box intersections and wrist trajectories extracted using FrankMocap [50].
- **VC-1** [31]: A visual encoder targeting a variety of embodied intelligence tasks, trained on large amounts of images using the Masked Auto-Encoder [19] paradigm.
- **VIP** [30]: A visual encoder for robotic manipulation using a novel implicit time contrastive objective.

For all methods based on ViT [12], namely DINO, HRP,

and MAPLE, we employ a ViT-B/16 [12] backbone. R3M and VIP use a ResNet50 [18] backbone. We initialize all baselines with their publicly available weights.

4.2. Evaluation in Simulation Environments

Existing Simulation Environments and Tasks. We first evaluate on the four tasks from the established DAPG [46] benchmark suite. The tasks consist of *open door*, *use hammer*, *rotate pen*, and *relocate ball* (see Figure 4).

New Environment and Task Designs. We further introduce a new set of four custom-designed, challenging evaluation tasks focused on dexterous manipulation that require more fine-grained control. Specifically, we design three tasks in the MuJoCo simulator [58] that require complex dexterous manipulation of tools using an Adroit hand [23]. All three tasks feature objects standing on a table, with the Adroit hand hovering above them and able to move freely (see Figure 4 for visualization). These tasks are (1) **Pan**: Lift a pan and place it on the induction plate of the nearby stove; (2) **Brush**: Retrieve the brush from a cup, then stroke it onto the nearby canvas; and (3) **Clothes Iron**: Grasp a clothes iron on a table by the handle and slide it along the leg of the pants positioned nearby. Detailed examples and renderings of the tasks are in Supp. Mat. Sec. S2.2.

Collect Demonstrations for the New Tasks. To conform with the established evaluation standard in literature [30, 39], we record 25 expert demonstrations for each of our tasks using Rokoko Smartgloves [48], employing the algorithm from [43] to retarget the human hand pose to the Adroit hand in the simulation. Please refer to the Supp. Mat. for details regarding the demonstration collection and the goal definitions of each task.

Experiment Protocol. Our evaluation uses both existing tasks as well as our newly designed ones. To evaluate a given visual encoder on a given task, MLP-based policies [39] are trained to control a robot hand in simulation given proprioceptive input and features from the visual encoder. Policies are evaluated every 1,000 out of a total of 20,000 training steps, with each evaluation consisting of 50 rollouts in the simulator. During each rollout, we randomize the position and/or rotation of the object(s). The policy receives a binary score indicating whether it fulfilled the intended task according to an environment-specific success criterion,

Model	Door	Hammer	Pen	Relocate	Brush	Iron	Pan	Mean
DINO [4]	19.6 $\pm$ 5.1	20.0 $\pm$ 0.9	60.4 $\pm$ 1.7	28.9 $\pm$ 0.8	5.6 $\pm$ 2.2	8.0 $\pm$ 1.9	4.2 $\pm$ 1.7	21.0
HRP [54]	17.3 $\pm$ 3.9	16.2 $\pm$ 2.5	61.8 $\pm$ 4.1	29.8 $\pm$ 0.8	4.2 $\pm$ 0.3	1.6 $\pm$ 1.1	1.1 $\pm$ 1.6	18.9
VC-1 [31]	14.7 $\pm$ 1.9	24.4 $\pm$ 4.6	56.7 $\pm$ 5.2	23.6 $\pm$ 0.8	4.4 $\pm$ 0.8	3.3 $\pm$ 0.5	0.4 $\pm$ 0.3	18.2
R3M [39]	15.3 $\pm$ 2.0	12.9 $\pm$ 3.6	60.0 $\pm$ 0.5	40.4 $\pm$ 1.3	7.3 $\pm$ 2.0	15.1 $\pm$ 1.1	0.2 $\pm$ 0.3	21.6
VIP [30]	11.1 $\pm$ 2.7	18.6 $\pm$ 4.3	66.4 $\pm$ 1.7	19.8 $\pm$ 1.3	5.1 $\pm$ 0.8	4.2 $\pm$ 1.4	2.4 $\pm$ 0.3	18.2
MAPLE-LN	21.6 $\pm$ 3.9	21.8 $\pm$ 1.9	65.6 $\pm$ 2.1	29.3 $\pm$ 2.5	5.8 $\pm$ 0.8	4.0 $\pm$ 1.1	3.6 $\pm$ 1.1	21.7
MAPLE-AP	22.0 $\pm$ 3.5	18.2 $\pm$ 4.9	63.8 $\pm$ 0.8	29.6 $\pm$ 3.0	4.2 $\pm$ 1.3	5.8 $\pm$ 2.3	0.9 $\pm$ 0.6	20.6

Table 1. **Results on Dexterous Simulation Environments.** We report the task completion success rate for seven simulation tasks, as well as the mean success rate. MAPLE-LN reaches the best mean performance on the evaluation suite, underscoring its excellent generalization ability. **Green** indicates the best performance, and **blue** the second best. We report the standard deviations across randomization seeds.

e.g. opening a door. The score of the encoder on the task is then the best average success rate across all 20 evaluations. Following prior work R3M [39], we calculate scores of encoders on tasks by additionally averaging the scores across three camera views, and three random seeds per view used to vary the policy network’s initialization, for a total of nine experiments per encoder and task. This averaging procedure helps improve the statistical robustness of the evaluation. For our custom-designed tasks, we employ a temporal horizon of 750 time steps. Gaussian noise is additionally added to the generated action to simulate imperfect execution and increase the tasks’ difficulty. See the Supp. Mat. for success definitions on our tasks.

Results. Table 1 compares our method MAPLE with various baselines and SotA methods on seven different dexterous manipulation tasks that require various levels of fine-grained control. We observe that MAPLE exhibits a strong generalization ability across tasks, and achieves the best mean performance score of all methods. MAPLE-LN outperforms MAPLE-AP for most tasks. The mean performance of R3M [39] is close to ours. However, it heavily relies on in-house human-annotated language captions during training (with a strong benefit in Relocate), while our method can learn in an entirely self-supervised manner. Surprisingly, the contrastive training objective of DINO [4] appears to give a strong performance, probably due to its strong semantic features that are robust to occlusion. The general low performance observed on our custom-designed tasks highlights the inherent complexity and challenges associated with tasks that require more complex dexterous skills and validates the usefulness of our newly introduced evaluation tasks.

Ablation Studies. We ablate the contribution of our contact and hand pose losses on the tasks used by [39] in Table 2, including DAPG (pen and relocate), as well as from the non-dexterous MetaWorld (5 tasks) [16, 65], and Franka Kitchen (5 tasks) [16] benchmarks.

Dexterous Tasks Benefit From $\mathcal{L}_{hand}$ And Its Tokenization. The dexterous tasks as well as the MetaWorld tasks benefit significantly from the inclusion of our hand pose loss, as evidenced by comparing the second row with the last row. Compared to a setting where the decoder recon-

$\mathcal{L}_{con,pt}$	$\mathcal{L}_{hand}$	DAPG	MetaWorld	FK
$\times$	$\checkmark$	44.0	73.6	55.5
$\checkmark$	$\times$	44.3	78.2	52.5
$\checkmark$	$\bar{T}$	44.0	81.6	55.2
$\square$	$\checkmark$	44.4	77.3	50.7
1	$\checkmark$	46.3	77.6	51.6
$\checkmark$	$\checkmark$	47.4	80.2	50.4

Table 2. **Ablation of Loss Terms on Dexterous and Non-Dexterous Environments.** We report the results of training our MAPLE-LN model. $\checkmark$ indicates the inclusion, while $\times$ indicates the exclusion of a given loss term. The $\mathcal{L}_{hand}$ loss without the hand tokenizer is indicated by $\bar{T}$. We further ablate sampling contact points randomly from hand-object box intersections ($\square$) or using only one contact prediction point (1).

structs raw hand joint configurations, using the hand pose tokenizer is helpful for the dexterous setting while harming the performance in the non-dexterous settings.

The Benefits of $\mathcal{L}_{con,pt}$. We notice that randomly sampling points from the intersection of hand and object bounding boxes hurts performance compared to our usage of the projected fingertips. Furthermore, reasoning about a single contact point (index finger only) for $\mathcal{L}_{con,pt}$ gives a worse performance on two out of three benchmark suites than reasoning about the index and thumb finger jointly.

4.3. Evaluation in Real-World Settings

Task Designs. Our real-world evaluation features two tasks derived from those used in simulation, as well as a novel task: (1) **Plush:** The robot picks up a plush object and places it on a tray, assessing the robustness to objects with a unique shape unseen during the encoder’s training. (2) **Pan:** The robot grasps the handle of a pan, lifts it, and places it onto an induction stove. After placement, the robot releases the handle. (3) **Brush:** The robot picks up a brush, moves it to a target frame, and performs a brushing motion on its surface before dropping the brush on a table. This is the most challenging task, testing both grasping precision and fine manipulation dexterity. Additional task descriptions and data collection details are in Supp. Mat. Sec. S1.

Hardware Setup. Our robotic setup comprises a Mimic robotic hand [35] mounted on a Franka Emika Panda

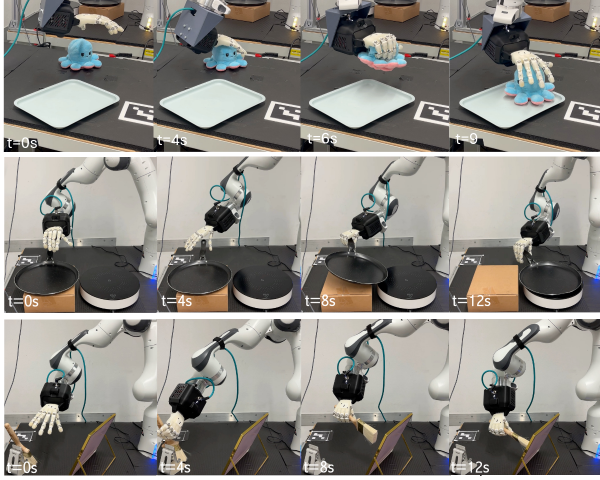


Figure 5. **Real world sequences of the evaluated tasks.** Example rollouts of MAPLE. From top to bottom: ‘Grab the plush animal (plush)’, ‘Place the pan’ (pan) and ‘paint the canvas’ (brush) tasks using the Mimic hand and the Franka Emika Panda manipulator.

robotic arm [17], along with two external cameras. Demonstrations are collected using Rokoko Smartgloves and the Coil Pro motion capture system [48], which provide proprioceptive location data from the demonstrator’s fingers and wrist. The external cameras record visual data.

Experiment Protocol. After training on egocentric videos, we freeze the encoder to ensure stable feature extraction and use the extracted features for manipulation policy training. For our real-world experiments, we employ the Diffusion Policy framework [5] to predict robot actions and use the Diffusion Transformer (DiT) [11] as the model architecture. Visual inputs are processed by our trained encoders, and their embeddings are combined with proprioceptive data. In particular, the end-effector pose and finger joint angles are transformed into tokens and concatenated with the image tokens before being passed into the DiT. We use DDIM [53] as our noise scheduler, with 100 steps during training and 8 steps at inference. Additional architectural and training details are provided in Supp. Mat. S1.2.

Results. For each model and task, we conducted 16 trials to evaluate success rates. Figure 6 summarizes the performance across all models. Our results indicate that our encoders consistently outperform alternative approaches. In particular, MAPLE-LN shows the highest overall performance, benefiting from its stability and strong generalizability across varying task conditions. Meanwhile, MAPLE-AP slightly outperforms MAPLE-LN on the most dexterous brush task. In contrast, other encoder-based methods exhibit behaviors that make them less suitable for our tasks: Using R3M [39] results in the policy overfitting and being less responsive to variations in initial conditions, leading to the lowest success rates. Using HRP [54] yields moderate performance on the pan task but induces abrupt

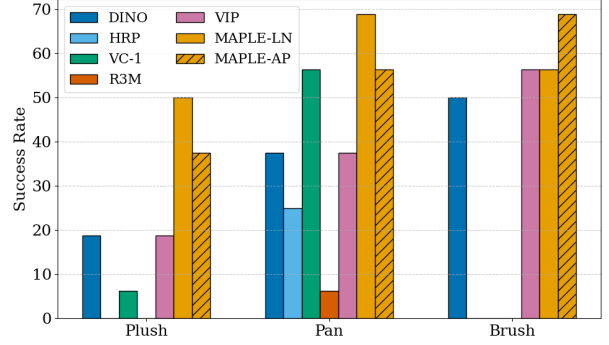


Figure 6. **Results on real-world environments.** We evaluate different encoders and use the extracted features for imitation learning. Missing bars correspond to a zero success rate.

hand and wrist movements, indicating challenges in accurate spatial reasoning. DINO [4] and VIP [30] offer competitive performance; however, they do not consistently match the stability and adaptability of MAPLE. VIP shows a slight edge over DINO on the brush task, but overall, both lag behind MAPLE in balancing generalizability and task-specific dexterity. VC-1 [31] presents unstable performance, performing well on the pan task yet struggling with the plush and brush tasks, seemingly insensitive to those tasks’ vision clues. Overall, our results demonstrate that MAPLE offers clear benefits in terms of both performance and adaptability not only in simulation but also in real-world manipulation tasks. Videos of our real-world experiments for all encoders are available in the Supp. Mat.

5. Conclusion

In this work, we study how to infuse dexterous manipulation priors into visual representations by training on diverse human videos, aiming to improve performance on downstream robotic tasks in human-centric environments. Common jaw grippers [1, 2, 7] limit interactions with a wide range of objects, making human-like hands more suitable for operating in human environments. We introduced MAPLE, a novel approach that learns manipulation-specific priors from ego-centric videos. MAPLE trains an encoder to predict hand-object contact points and detailed 3D hand poses from a single image, producing features useful for downstream manipulation tasks. For the benefit of the community and a more rigorous evaluation, we further propose new dexterous simulation environments. Our strong results on both new tasks and existing benchmark tasks in simulation, as well as our superior real-world performance demonstrate promising manipulation prior learning and excellent generalization ability with MAPLE. Future directions include integrating MAPLE into real-world control loops and incorporating more diverse, possibly exocentric human manipulation datasets.

References

- [1] Shikhar Bahl, Russell Mendonca, Lili Chen, Unnat Jain, and Deepak Pathak. Affordances From Human Videos as a Versatile Representation for Robotics. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13778–13790, 2023. 3, 4, 8
- [2] Homanga Bharadhwaj, Abhinav Gupta, Shubham Tulsiani, and Vikash Kumar. Zero-Shot Robot Manipulation from Passive Human Videos. In *ICRA2023 Workshop on Pretraining for Robotics (PT4R)*, 2023. 3, 8
- [3] Kaylee Burns, Zach Witzel, Jubayer Ibn Hamid, Tianhe Yu, Chelsea Finn, and Karol Hausman. What Makes Pre-Trained Visual Representations Successful for Robust Manipulation? *arXiv preprint arXiv:2312.12444*, 2023. 2
- [4] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging Properties in Self-Supervised Vision Transformers. In *Proceedings of the International Conference on Computer Vision (ICCV)*, 2021. 2, 5, 6, 7, 8
- [5] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, page 02783649241273668, 2023. 3, 8
- [6] Sammy Christen, Muhammed Kocabas, Emre Aksan, Jemin Hwangbo, Jie Song, and Otmar Hilliges. D-Grasp: Physically Plausible Dynamic Grasp Synthesis for Hand-Object Interactions. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20577–20586, 2022. 2
- [7] Open X-Embodiment Collaboration. Open X-Embodiment: Robotic Learning Datasets and RT-X Models. <https://arxiv.org/abs/2310.08864>, 2023. 8
- [8] Dima Damen, Hazel Doughty, Giovanni Maria Farinella, Antonino Furnari, Jian Ma, Evangelos Kazakos, Davide Moltisanti, Jonathan Munro, Toby Perrett, Will Price, and Michael Wray. Rescaling Egocentric Vision: Collection, Pipeline and Challenges for EPIC-KITCHENS-100. *International Journal of Computer Vision (IJCV)*, 130:33–55, 2022. 2
- [9] Ahmad Darkhalil, Dandan Shan, Bin Zhu, Jian Ma, Amlan Kar, Richard Higgins, Sanja Fidler, David Fouhey, and Dima Damen. EPIC-KITCHENS VISOR Benchmark: Video Segmentations and Object Relations. In *Proceedings of the Neural Information Processing Systems (NeurIPS) Track on Datasets and Benchmarks*, 2022. 4, 15, 16
- [10] Sudeep Dasari, Mohan Kumar Srirama, Unnat Jain, and Abhinav Gupta. An Unbiased Look at Datasets for Visuomotor Pre-Training. In *Conference on Robot Learning*, pages 1183–1198. PMLR, 2023. 2
- [11] Sudeep Dasari, Oier Mees, Sebastian Zhao, Mohan Kumar Srirama, and Sergey Levine. The ingredients for robotic diffusion transformers. *arXiv preprint arXiv:2410.10088*, 2024. 3, 8, 12
- [12] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, G Heigold, S Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2020. 5, 6
- [13] Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4RL: Datasets for Deep Data-Driven Reinforcement Learning, 2020. 3
- [14] Kristen Grauman, Andrew Westbury, Eugene Byrne, Zachary Chavis, Antonino Furnari, Rohit Girdhar, Jackson Hamburger, Hao Jiang, Miao Liu, Xingyu Liu, et al. Ego4D: Around the World in 3000 Hours of Egocentric Video. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 18995–19012, 2022. 2, 6, 15
- [15] Kristen Grauman, Andrew Westbury, Lorenzo Torresani, Kris Kitani, Jitendra Malik, Triantafyllos Afouras, Kumar Ashutosh, Vijay Baiyya, Siddhant Bansal, Bikram Boote, et al. Ego-Exo4D: Understanding Skilled Human Activity from First- and Third-Person Perspectives. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 19383–19400, 2024. 2
- [16] Abhishek Gupta, Vikash Kumar, Corey Lynch, Sergey Levine, and Karol Hausman. Relay Policy Learning: Solving Long-Horizon Tasks via Imitation and Reinforcement Learning. *arXiv preprint arXiv:1910.11956*, 2019. 3, 7, 13
- [17] Sami Haddadin, Sven Parusel, Lars Johannsmeier, Saskia Golz, Simon Gabl, Florian Walch, Mohamadreza Sabaghian, Christoph Jähne, Lukas Hausperger, and Simon Haddadin. The Franka Emika Robot: A Reference Platform for Robotics Research and Education. *IEEE Robotics & Automation Magazine*, 29(2):46–64, 2022. 8, 12, 13
- [18] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016. 6
- [19] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked Autoencoders Are Scalable Vision Learners. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 16000–16009, 2022. 6
- [20] Zeng Jia, Bu Qingwen, Wang Bangjun, Xia Wenke, Chen Li, Dong Hao, Song Haoming, Wang Dong, Hu Di, Luo Ping, Cui Heming, Zhao Bin, Li Xuelong, Qiao Yu, and Li Hongyang. Learning manipulation by predicting interaction. In *Proceedings of Robotics: Science and Systems (RSS)*, 2024. 3
- [21] Simar Kareer, Dhruv Patel, Ryan Punamiya, Pranay Mathur, Shuo Cheng, Chen Wang, Judy Hoffman, and Danfei Xu. Egomimic: Scaling imitation learning via egocentric video. *arXiv preprint arXiv:2410.24221*, 2024. 3
- [22] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. Openvla: An open-source vision-language-action model, 2024. 3
- [23] Vikash Kumar, Zhe Xu, and Emanuel Todorov. Fast, Strong and Compliant Pneumatic Actuation for Dexterous Tendon-Driven Hands. In *2013 IEEE International Conference on*

- Robotics and Automation*, pages 1512–1519. IEEE, 2013. 3, 6, 14
- [24] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer Normalization. *ArXiv e-prints*, pages arXiv–1607, 2016. 5
- [25] Gen Li, Kaifeng Zhao, Siwei Zhang, Xiaozhong Lyu, Mihai Dusmanu, Yan Zhang, Marc Pollefeys, and Siyu Tang. EgoGen: An Egocentric Synthetic Data Generator. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14497–14509, 2024. 2
- [26] Liunian Harold Li, Pengchuan Zhang, Haotian Zhang, Jianwei Yang, Chunyuan Li, Yiwu Zhong, Lijuan Wang, Lu Yuan, Lei Zhang, Jenq-Neng Hwang, Kai-Wei Chang, and Jianfeng Gao. Grounded Language-Image Pre-Training. In *CVPR*, 2022. 16
- [27] Toru Lin, Yu Zhang, Qiyang Li, Haozhi Qi, Brent Yi, Sergey Levine, and Jitendra Malik. Learning visuotactile skills with two multifingered hands. *arXiv preprint arXiv:2404.16823*, 2024. 3
- [28] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, et al. Grounding DINO: Marrying Dino With Grounded Pre-training for Open-Set Object Detection. *arXiv preprint arXiv:2303.05499*, 2023. 16
- [29] I Loshchilov. Decoupled Weight Decay Regularization. *arXiv preprint arXiv:1711.05101*, 2017. 6
- [30] Yecheng Jason Ma, Shagun Sodhani, Dinesh Jayaraman, Osbert Bastani, Vikash Kumar, and Amy Zhang. Vip: Towards universal visual reward and representation via value-implicit pre-training. In *The Eleventh International Conference on Learning Representations*, 2022. 2, 3, 6, 7, 8
- [31] Arjun Majumdar, Karmesh Yadav, Sergio Arnaud, Jason Ma, Claire Chen, Sneha Silwal, Aryan Jain, Vincent-Pierre Berges, Tingfan Wu, Jay Vakil, et al. Where Are We in the Search for an Artificial Visual Cortex for Embodied Intelligence? *Advances in Neural Information Processing Systems*, 36:655–677, 2023. 2, 3, 5, 6, 7, 8
- [32] Rayhane Mama, Marc S Tyndel, Hashiam Kadhim, Cole Clifford, and Ragavan Thurairatnam. NWT: Towards Natural Audio-To-Video Generation With Representation Learning. *arXiv preprint arXiv:2106.04283*, 2021. 5
- [33] Priyanka Mandikal and Kristen Grauman. Learning Dexterous Grasping With Object-Centric Visual Affordances. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6169–6176. IEEE, 2021. 3
- [34] Priyanka Mandikal and Kristen Grauman. DexVIP: Learning Dexterous Grasping with Human Hand Pose Priors from Video. In *Conference on Robot Learning*, pages 651–661, 2022. 3
- [35] mimic Robotics. mimic P0.4 Robotic Hand. <https://www.mimicrobotics.com/>, 2025. 3, 7, 12
- [36] David Mizrahi, Roman Bachmann, Oğuzhan Fatih Kar, Teresa Yeo, Mingfei Gao, Afshin Dehghan, and Amir Zamir. 4M: Massively Multimodal Masked Modeling. In *Advances in Neural Information Processing Systems*, 2023. 5
- [37] MuJoCo Maintainers. MuJoCo XML Reference. <https://mujoco.readthedocs.io/en/stable/XMLreference.html>, 2024. 15
- [38] Tushar Nagarajan, Christoph Feichtenhofer, and Kristen Grauman. Grounded human-object interaction hotspots from video. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 8688–8697, 2019. 3
- [39] Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3M: A Universal Visual Representation for Robot Manipulation. In *6th Annual Conference on Robot Learning*, 2022. 2, 3, 5, 6, 7, 8, 15
- [40] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018. 5
- [41] Georgios Pavlakos, Dandan Shan, Ilija Radosavovic, Angjoo Kanazawa, David Fouhey, and Jitendra Malik. Reconstructing Hands in 3D with Transformers. In *CVPR*, 2024. 4, 5, 15
- [42] Haozhi Qi, Ashish Kumar, Roberto Calandra, Yi Ma, and Jitendra Malik. In-hand object rotation via rapid motor adaptation. In *Conference on Robot Learning*, pages 1722–1732. PMLR, 2023. 2
- [43] Qin, Yuzhe, Yueh-Hua Wu, Shaowei Liu, Hanwen Jiang, Ruihan Yang, Yang Fu, and Xiaolong Wang. DexMV: Imitation Learning for Dexterous Manipulation From Human Videos, 2021. 3, 6, 13
- [44] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning Transferable Visual Models From Natural Language Supervision. In *International Conference on Machine Learning*, pages 8748–8763. PMLR, 2021. 2, 5
- [45] Ilija Radosavovic, Baifeng Shi, Letian Fu, Ken Goldberg, Trevor Darrell, and Jitendra Malik. Robot Learning with Sensorimotor Pre-Training. In *Conference on Robot Learning*, pages 683–693. PMLR, 2023. 2
- [46] Aravind Rajeswaran, Vikash Kumar, Abhishek Gupta, Giulia Vezzani, John Schulman, Emanuel Todorov, and Sergey Levine. Learning Complex Dexterous Manipulation With Deep Reinforcement Learning and Demonstrations. In *Proceedings of Robotics: Science and Systems (RSS)*, 2018. 2, 3, 6, 15
- [47] Frano Rajič, Lei Ke, Yu-Wing Tai, Chi-Keung Tang, Martin Danelljan, and Fisher Yu. Segment Anything Meets Point Tracking. *arXiv:2307.01197*, 2023. 4, 15
- [48] Rokoko ApS. SmartGloves – Affordable Quality and Hand Motion Capture. <https://www.rokoko.com/products/smartgloves>, 2024. 6, 8, 12
- [49] Javier Romero, Dimitrios Tzionas, and Michael J. Black. Embodied Hands: Modeling and Capturing Hands and Bodies Together. *ACM Transactions on Graphics, (Proc. SIGGRAPH Asia)*, 36(6), 2017. 4, 5
- [50] Yu Rong, Takaaki Shiratori, and Hanbyul Joo. FrankMocap: A Monocular 3D Whole-Body Pose Estimation System via Regression and Integration. In *IEEE International Conference on Computer Vision Workshops*, 2021. 6
- [51] J.P. Serra. *Image Analysis and Mathematical Morphology*. Academic Press, 1982. 4
- [52] Himanshu Gaurav Singh, Antonio Loquercio, Carmelo Sferazza, Jane Wu, Haozhi Qi, Pieter Abbeel, and Jitendra Ma-

- lik. Hand-Object Interaction Pretraining from Videos. *arXiv preprint arXiv:2409.08273*, 2024. [2](#)
- [53] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. [8](#)
- [54] Mohan Kumar Srirama, Sudeep Dasari, Shikhar Bahl, and Abhinav Gupta. HRP: Human Affordances for Robotic Pre-Training. In *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, 2024. [2](#), [3](#), [4](#), [5](#), [6](#), [7](#), [8](#)
- [55] Andrew Szot, Alexander Clegg, Eric Undersander, Erik Wijmans, Yili Zhao, John Turner, Noah Maestre, Mustafa Mukadam, Devendra Singh Chaplot, Oleksandr Maksymets, et al. Habitat 2.0: Training Home Assistants to Rearrange Their Habitat. *Advances in neural information processing systems*, 34:251–266, 2021. [3](#)
- [56] Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, et al. DeepMind Control Suite. *arXiv preprint arXiv:1801.00690*, 2018. [3](#)
- [57] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, Jianlan Luo, You Liang Tan, Lawrence Yunliang Chen, Pannag Sanketi, Quan Vuong, Ted Xiao, Dorsa Sadigh, Chelsea Finn, and Sergey Levine. Octo: An open-source generalist robot policy, 2024. [3](#)
- [58] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A Physics Engine for Model-Based Control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. [6](#)
- [59] Dylan Turpin, Tao Zhong, Shutong Zhang, Guanglei Zhu, Eric Heiden, Miles Macklin, Stavros Tsogkas, Sven Dickinson, and Animesh Garg. Fast-grasp’d: Dexterous multi-finger grasp generation through differentiable simulation. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 8082–8089. IEEE, 2023. [2](#)
- [60] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30, 2017. [6](#)
- [61] Ruicheng Wang, Jialiang Zhang, Jiayi Chen, Yinzhen Xu, Puhao Li, Tengyu Liu, and He Wang. Dexgraspnet: A large-scale robotic dexterous grasp dataset for general objects based on simulation. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 11359–11366. IEEE, 2023. [2](#)
- [62] Xin Wang, Taein Kwon, Mahdi Rad, Bowen Pan, Ishani Chakraborty, Sean Andrist, Dan Bohus, Ashley Feniello, Bugra Tekin, Felipe Vieira Frujeri, et al. HoloAssist: an Ego-centric Human Interaction Dataset for Interactive AI Assistants in the Real World. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 20270–20281, 2023. [2](#)
- [63] Manuel Wüthrich, Felix Widmaier, Felix Grimmering, Joel Akpo, Shruti Joshi, Vaibhav Agrawal, Bilal Hammoud, Majid Khadiv, Miroslav Bogdanovic, Vincent Berenz, et al. TriFinger: An Open-Source Robot for Learning Dexterity. *arXiv preprint arXiv:2008.03596*, 2020. [3](#)
- [64] Tete Xiao, Ilija Radosavovic, Trevor Darrell, and Jitendra Malik. Masked visual pre-training for motor control. *arXiv preprint arXiv:2203.06173*, 2022. [3](#)
- [65] Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-World: A Benchmark and Evaluation for Multi-Task and Meta Reinforcement Learning. In *Conference on Robot Learning*, pages 1094–1100. PMLR, 2020. [3](#), [7](#), [13](#)
- [66] Hui Zhang, Sammy Christen, Zicong Fan, Otmar Hilliges, and Jie Song. GraspXL: Generating grasping motions for diverse objects at scale. In *European Conference on Computer Vision*, pages 386–403. Springer, 2024. [2](#)
- [67] Hui Zhang, Sammy Christen, Zicong Fan, Luocheng Zheng, Jemin Hwangbo, Jie Song, and Otmar Hilliges. Artigrasp: Physically plausible synthesis of bi-manual dexterous grasping and articulation. In *2024 International Conference on 3D Vision (3DV)*, pages 235–246. IEEE, 2024. [2](#)
- [68] Tony Z Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023. [3](#)

MAPLE: Encoding Dexterous Robotic Manipulation Priors Learned From Egocentric Videos

Supplementary Material

Contents

S1. Evaluation Details in Real World	12
S1.1 Robot Platform and Policy Execution	12
S1.2 Imitation Learning Details	12
S1.3 Plush Pick and Place	12
S1.4 Pan Pick and Place	13
S1.5 Brush Frames	13
S2. Demonstration Data and Benchmark Environments for Simulation	13
S2.1 Hand Retargeting Algorithm	13
S2.2 Visualization of Demonstration Trajectories .	13
S3. Evaluation Details in Simulation	13
S3.1 Environment Success Criteria	15
S3.2 Limitations	15
S4. Training Supervision Extraction	15
S4.1 Error Analysis of Contact Supervision Extraction	15
S4.2 Error Analysis of Prediction Frame Search .	16
S4.3 Tokenizer Visualizations	17

S1. Evaluation Details in Real World

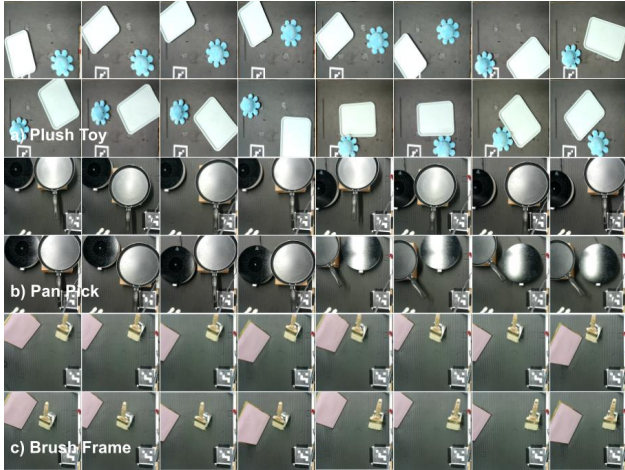


Figure S7. The initial setups of all real-world evaluations: plush toy pick and place, pan pick and place, and brush frame. Each method is evaluated for 16 episodes, all starting from the states shown in the figure.

In this section, we provide more details on the imitation learning evaluations introduced in Sec. 4.3. Each method is

evaluated for 16 episodes for every task, with a predefined initial setup as shown in Fig. S7.

S1.1. Robot Platform and Policy Execution

Our system consists of a Mimic robotic hand [35] mounted on a Franka Emika Panda robotic arm [17]. Two external OAK-D cameras capture the scene, and the arm and hand are controlled at 10 Hz. The arm’s action is defined by a 7-dimensional end-effector pose (x, y, z , and a quaternion), and the hand’s action is represented by a 16-dimensional vector corresponding to its joint angles. All camera images are cropped and resized to 224×224 pixels. Our framework uses an observation window spanning two time steps, while control actions are executed in chunks spanning 10 time steps.

S1.2. Imitation Learning Details

To accelerate training and enable larger batch sizes, we cache embeddings from the observation encoders directly within the dataset. This approach significantly speeds up training. Our DiT [11] backbone has a model dimension of 512, consists of 6 blocks, and employs 8 attention heads. The network is trained using the Adam optimizer with a cosine warmup learning rate schedule. All methods are evaluated after 35,000 training steps.

S1.3. Plush Pick and Place

Demonstrations. 101 demonstrations are collected and used for training. 80% of them are used as training set, 20% as evaluation set. The starting condition is randomized. Demonstrations are collected by an experienced operator wearing Rokoko Smartgloves and the Coil Pro motion capture setup [48], which provides proprioceptive location data of the demonstrator’s fingers and wrist. Images from three cameras are collected and fed during policy rollout: side camera, front camera and wrist camera.

Evaluation. Each method is evaluated for 16 episodes, all starting from the same set of initial conditions as shown in Fig. S7.a. For each trial and task, the robot starts from the nominal pose. The episode is counted as successful only if the toy is grasped and fully placed in the tray. A second attempt is allowed if the policy misses the first grab and successfully grabs the toy without human intervention within the time limit of 30 seconds.

S1.4. Pan Pick and Place

Demonstrations. 446 demonstrations are collected and used for training. 80% of them are used as training set, 20% as evaluation set. The starting condition is randomized. The demonstrations are collected with the same Rokoko solution as described before. Images from two cameras are collected and fed during policy rollout: side camera and front camera.

Evaluation. Each method is evaluated for 16 episodes, all starting from the same set of initial conditions as shown in Fig. S7.b. For each trial and task, the robot starts from the nominal pose. The episode is counted as successful only if the pan is grasped, lifted, placed on the induction stove and touches nothing else.

S1.5. Brush Frames

Demonstrations. 226 demonstrations are collected and used for training. 80% of them are used as training set, 20% as evaluation set. The starting condition is randomized. The demonstrations are collected using the same Rokoko solution as described above. Images from two cameras are collected and fed during the policy rollout: side camera and front camera.

Evaluation. Each method is evaluated for 16 episodes, all starting from the same set of initial conditions as shown in Fig. S7.c. For each trial and task, the robot starts from the nominal pose. The episode is counted as successful only if the brush is successfully grasped, touches the board, and is then dropped on the table.

S2. Demonstration Data and Benchmark Environments for Simulation

S2.1. Hand Retargeting Algorithm

The algorithm from [43] works by optimizing over the vector of the Adroit joint actuations, minimizing a Huber loss on the difference between pairs of an inter-fingertip or finger-to-wrist vector on the human hand and a corresponding vector measuring the same distance on the robotic hand. The hand pose is inferred by the aforementioned retargeting procedure, while the global position of the wrist joint is obtained using the Rokoko Coil Pro, an EMF-based tracking device. We use virtual PID controllers to drive each joint.

S2.2. Visualization of Demonstration Trajectories

We show selected frames from our demonstration trajectories for the brush (Figure S8), the iron (Figure S9) and the pan (Figure S10) tasks for each camera view.

S3. Evaluation Details in Simulation

To evaluate the suitability of a given visual encoder for downstream dexterous manipulation tasks, we use simulators in which a policy network consuming features from a visual encoder is trained to operate an observed robotic manipulator. As can also be expected from a real-world deployment, the policy network additionally receives proprioceptive features describing the current joint configurations of the robot as input. Position and orientation information related to the object(s) to be manipulated is only provided through visual cues, so as to increase realism and encourage the use of visual features to grasp and manipulate the object. The Franka Kitchen [16] and Metaworld [65] environments feature Franka Emika Panda [17] arms (7 DoF) with jaw grippers, exhibiting 7+2 DoF, while the DAPG and

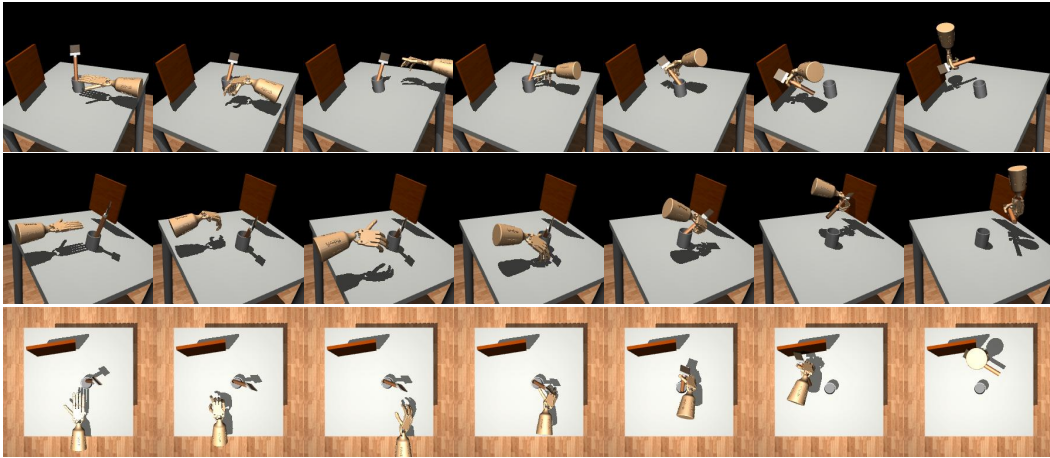


Figure S8. **Example Demonstration Trajectory:** Camera observations of a demonstration for the 'Brush' task. The top row displays the left camera view, the middle row shows the right camera view, and the bottom row presents the top camera view.

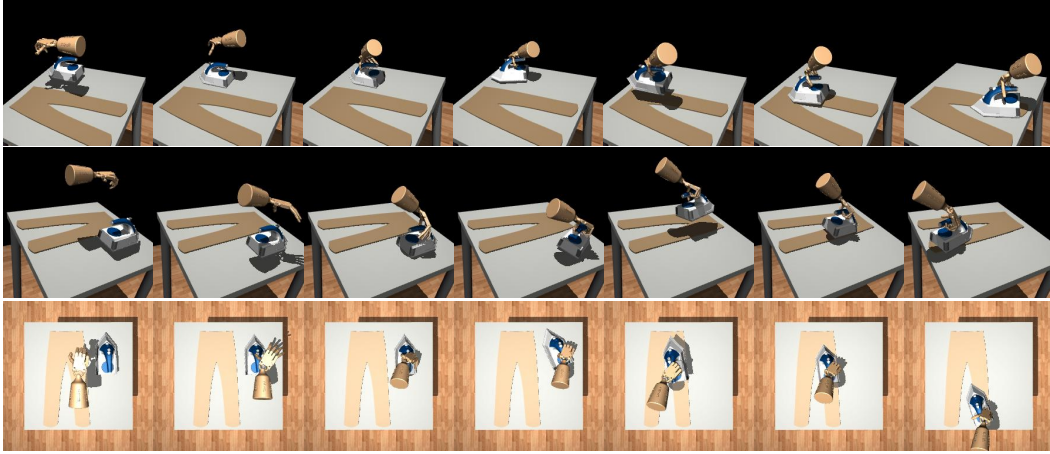


Figure S9. **Example Demonstration Trajectory:** Camera observations of a demonstration for the 'Iron' task. The top row displays the left camera view, the middle row shows the right camera view, and the bottom row presents the top camera view.

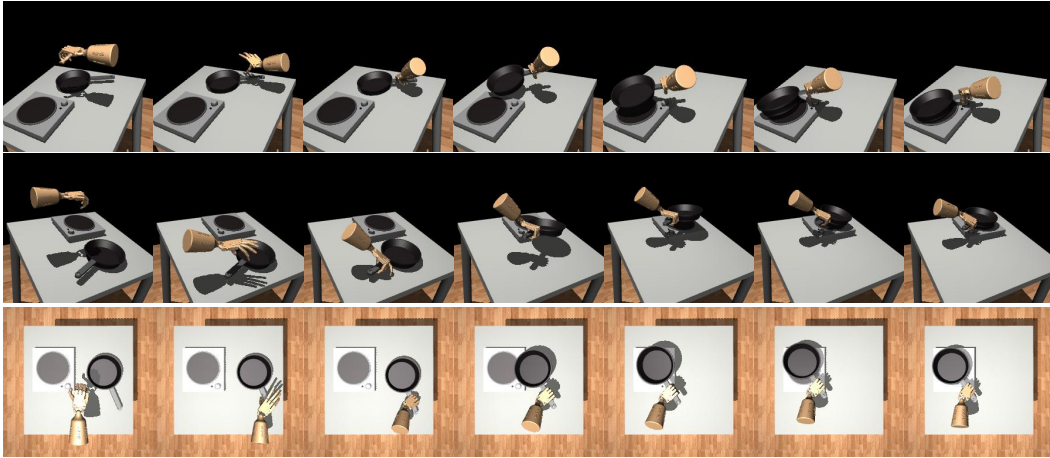


Figure S10. **Example Demonstration Trajectory:** Camera observations of a demonstration for the 'Pan' task. The top row displays the left camera view, the middle row shows the right camera view, and the bottom row presents the top camera view.

our dexterous environments feature the Adroit [23] dexterous hand with 30 DoF in total.

The policy networks are trained to perform the respective tasks using demonstrations recorded by human participants, as described in Sec. 4.2. While training the policy networks, we periodically perform rollouts (i.e. simulation runs where the performance of the policy is assessed). The policies are trained for 20,000 training iterations. Rollouts are performed every 1,000 training iterations, and the highest success rate achieved over a given run is chosen for the computation of the final average success rates reported in the tables. 50 rollouts and 2 views are used during each evaluation for the experiments on the Franka Kitchen environments in Table 2. Note that the MetaWorld and DAPG environments, as well as our dexterous environments randomly set the orientation and position of the object(s) to

manipulate upon each simulation restart to make the tasks more challenging and realistic.

During the training of the policy, the checkpoint of the respective visual encoder is used as a frozen feature extractor, and only the parameters of the policy are trained. For our MAPLE-LN model, we use the checkpoint obtained after training 3,000 iterations on our Ego4D-based dataset. For MAPLE-AP, we use the checkpoint obtained after 2,000 iterations. These iteration counts were chosen to match the approximate contact prediction performance saturation point on a small validation dataset, manually constructed from prediction frames produced by our data extraction pipeline. In all simulation experiments, the policy is a 2-layer MLP with 256 units for each hidden layer. The MLP receives the visual features produced by the respective encoder from a rendered RGB view of the input

scene. Examples of rendered simulator views are visualized in [Figure S8](#), [Figure S9](#), [Figure S10](#). The MLP further receives a number p of proprioceptive features encoding the current joint configuration of the jaw gripper resp. robotic hand. Specifically, we use $p = 9$ for all Franka Kitchen environments and $p = 4$ for all Metaworld environments. We use $p = 24$ for the DAPG pen environment, as well as $p = 30$ for the DAPG relocate, hammer and door environments. These values are chosen to match the setup in previous work [39]. We further set $p = 30$ for all our dexterous environments. The number of proprioceptive features generally corresponds to the DoF of the robotic manipulator. At each simulation timestep, given these environment observations, the MLP networks output actuation vectors which determine the force applied at each actuator according to the conversion outlined in [37]. To learn from human demonstrations, the policy is provided observation inputs recorded during human teleoperation of the robot manipulator, and is trained to predict the actuation vector corresponding to the retargeted action that the human performed at that timestep. As is standard in literature [39, 46], we train a separate policy per environment.

S3.1. Environment Success Criteria

Here, we provide detailed criteria that must be fulfilled for a task to count as successfully completed, for each of our dexterous environments. For distance measurements, MuJoCo site markers [37] attached to the robotic hand, manipulated object (*brush*, *pan*, *iron*) and target object (*wooden board*, *induction stove*, *clothes*) are used.

- **Pan:** The bottom of the pan must be in close vicinity of the induction stove. Here, we explicitly do not require the hand to contact the pan’s handle to permit a (slight) dropping of the pan to the induction stove to achieve the defined goal.
- **Brush:** The tip of the brush must contact the wooden board’s front side while the palm of the hand is in the close vicinity of the handle of the brush.
- **Clothes Iron:** The bottom of the clothes iron must slide along at least three of eight markers placed equidistantly along the right leg of the pants laid out on the table, while the palm of the hand is in close vicinity of the handle of the clothes iron.

S3.2. Limitations

Our work only considers a single embodiment in simulation and a single embodiment in the real-world experiments, making it difficult to estimate the impact of the particular robot on the performance achievable with our method. Further, the training data extracted using our pipeline exhibits noise (see [subsection S4.1](#), [subsection S4.2](#)) that is likely harmful during training of the encoder.

S4. Training Supervision Extraction

To obtain data for training our visual encoder, we start by processing every frame in the `clips` subset of the Ego4D dataset [14] using the VISOR-HOS [9] hand-object interaction segmenter, and retain frames where exactly one right hand is contacting an object with an HOS confidence at least $c \geq 0.9$, and no left hand is contacting an object with a similarly high confidence. Afterwards, we obtain frame-wise HaMeR [41] hand reconstructions for each contact frame. These hand reconstructions are used to initialize the prediction-frame-seeking contact point tracking with the thumb and index fingertips: for each contact frame, we first perform a binary erosion operation (12 iterations) on the object mask to remove the boundary, as motivated in Section 3.1. Then, we project the thumb and index fingertip points of the right hand in the contact frame to the eroded object mask. In case the ratio of the projected points’ Euclidean distance to their original distance falls outside the range between 0.3 and 1.7, the contact frame is not used. This helps eliminate objects with degenerate HOS masks. Otherwise, we initialize SAM-PT [47] with the projected fingertip points, as well as 10 other points sampled from the eroded object mask according to the query point sampling algorithm described in [47] and used to track the object. We continue tracking the points backward through the video until we encounter a frame where the HOS hand mask dilated using 75 binary dilation iterations (i.e., the expanded hand mask) no longer intersects the backtracked object mask. In case no such frame is encountered after 45 frames (i.e. 1.5s), the given contact frame is discarded. Otherwise, we use this frame as the prediction frame, and the backtracked points in that frame as training supervision for the contact loss. For the hand pose loss, we note that there are few constraints on the hand pose as it is moving in the direction of an object to grasp it. Hence, we always let our method reconstruct the hand pose at the contact frame, to promote learning the more informative grasping pose expected to be seen when the hand is already in contact with the object.

Examples of prediction frames and contact point labels extracted using our pipeline are provided in [Figure S11](#).

S4.1. Error Analysis of Contact Supervision Extraction

Although MAPLE achieves strong performance using dexterous manipulation priors learned from egocentric videos, occasionally we still notice some cases where our contact pseudo-labels are not accurate. Here, we analyze several failure modes of our automatic supervision extraction pipeline, as described in Sec. 3.1. The elimination of these errors has the potential to improve the effectiveness of the performance further and is left as a direction for future work.

Self-Contact. The first type of categorized error is *self-*

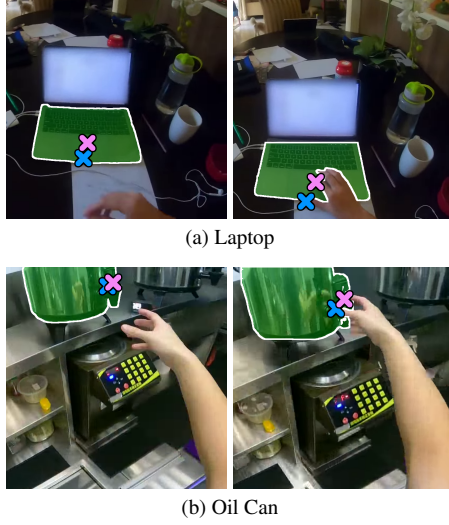


Figure S11. **Object Mask and Contact Points.** The left column displays the prediction frames, while the right column shows the corresponding contact frames automatically extracted during our dataset acquisition step. The **cyan** cross mark represents the contact point of the thumb, and the **pink** cross mark indicates the contact point of the index finger.

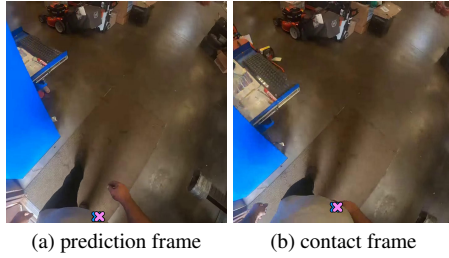


Figure S12. **Self-Contact Error.** The right hand is hovering above the actor’s shirt in (b), which is detected as a contact frame by the HOS segmenter.

contact, which occurs when the hand makes contact with the human body or worn clothing rather than an interacted object. In such cases, since the hand still appears to establish contact with a surface in the input image, the hand-object interaction segmenter [9] may misclassify these frames as contact frames. An example of a self-contact error is illustrated in Figure S12. A large proportion of training data exhibiting self-contact errors may induce an undesirable bias towards human garments in the encoder’s features. We hypothesize that this type of error can be tackled effectively by prompting visual grounding models [26, 28] for masks of people in the prediction frame and eliminating samples with contact points inside such masks.

Premature Contact. Another type of error associated with contact frames occurs when the hand is still approaching an object rather than making contact, yet the HOS segmenter



Figure S13. **Premature Contact Error.** The actor is reaching out for the garden scissors in (a), yet the hand is already detected as being in contact with the jug by the HOS segmenter, resulting in erroneous contact points in the prediction frame.

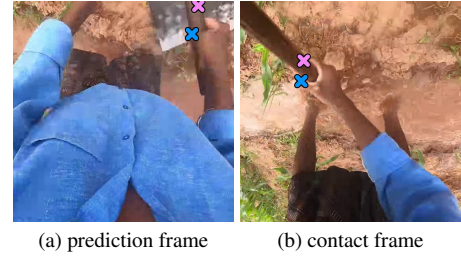


Figure S14. **Premature Termination Error.** In the prediction frame, the hand is still in contact with the stick.

identifies the presence of a hand-object interaction with either a background object or a border region of the target object. We classify these instances as *premature contact*. Figure S13 illustrates an example of this error type. Eliminating premature contact errors may prove nontrivial, as we are not provided depth information in our setting, and as the human touch itself can often be fleeting, such as when pressing a button. A large proportion of training data exhibiting premature contact errors may shift the encoder’s focus away from regions of interaction towards background objects and irrelevant regions of the interacted object.

S4.2. Error Analysis of Prediction Frame Search

Premature Tracking Termination. When tracking the contact points back in time from a contact frame in search for a prediction frame, the hand is sometimes not accurately detected due to a failure of the HOS segmenter, where it may be mistakenly identified as not being near the object region even though it is still contacting the object. For such samples, the prediction frame will still feature the hand interacting with the object and predicting the contact points will most frequently reduce to regressing the position of the thumb and index finger in the frame. A large proportion of training data exhibiting premature tracking termination errors may train the encoder and decoder to simply regress fingertip positions for the contact points, which will result in less informative features when a manipulator in a down-

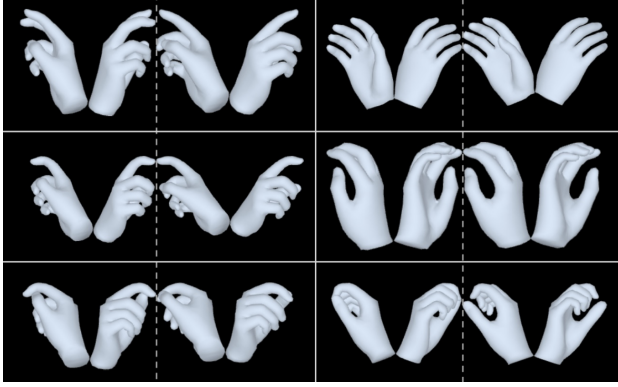


Figure S15. **Original and Tokenized Hand Meshes.** The reconstructions (right of dashed lines) differ only slightly from the original input (left of dashed lines), preserving the original grasp.

stream application is presented with an image of a yet ungrasped object. An illustration of this error type is provided in [Figure S14](#).

S4.3. Tokenizer Visualizations

We present examples of direct comparisons between original hand poses as regressed from Ego4D, and the hand poses after tokenization, in [Figure S15](#). We find that our tokenizer learns to tokenize the hand poses well in most cases, with only small deviations observable between the original and the reconstructed data after tokenization.